Amazon Customer Sentiment Analysis

IE 7374: Data Pipeline Assignment Group
Number: 7

Team Members:

Aniruthan Swaminathan Arulmurugan
Manikandan Mohan
Manivannan Senthilkumar
Janani Karthikeyan
Kiran Tamilselvan
Aravind Anbazhagan

Introduction

Business Problem:

Understanding customer sentiment is crucial for businesses to enhance user experience, improve product offerings, and make data-driven decisions. This project focuses on analyzing Amazon customer reviews to classify them into positive, neutral, or negative sentiments using machine learning and natural language processing techniques.

Challenges:

- **Large-scale Data Processing:** Handling 54.41 GB of Amazon US Customer Reviews Dataset.
- **Data Imbalance:** Unequal sentiment distribution may lead to biased model predictions.
- **Automation & Scalability:** Ensuring real-time processing with a scalable pipeline.
- **Bias & Ethical Concerns:** Avoiding potential model biases and ensuring fairness.

Datasource

- **Dataset Name:** Amazon US Customer Reviews
- **Format:** Tab-Separated Values (.tsv)
- **Key Fields:** review_id, product_id, star_rating, review_body, review_date
- **Source:** Amazon US Customer Reviews on Kaggle

Installation & Setup

Prerequisites:

- Python >= 3.9
- Git
- Docker (for containerization)
- Google Cloud Platform (GCP) account for cloud storage
- Apache Airflow for workflow automation

User Installation

The User Installation Steps are as follows:

1. **Clone the Repository:**
   git clone https://github.com/Aniruthan-0709/Sentiment-Analysis-Tool-for-Product-Reviews
   cd Sentiment-Analysis-Tool-for-Product-Reviews

2. **Check System Requirements:**

- Ensure your system meets the following prerequisites:

  OS Compatibility: Windows, Mac, Linux
  Python Version: >=3.9
  Sufficient Memory Allocation for Docker: Minimum 8GB RAM required

- Docker Installed: Verify Docker installation using:
  docker --version

- Docker Compose Installed: Verify with:
  docker-compose --version

- Increase Docker's memory allocation if needed via Docker Desktop > Settings > Resources

3. **Configure Environment Variables:**

- Place your Google Cloud Service Account Key (key.json) in the config/ folder.
- Update the .env file with:
  GCP_BUCKET=mlops_dataset123
  SOURCE_BLOB=mlops_dataset123/data/raw/Sampled_11_Product_Categories.csv
  GOOGLE_APPLICATION_CREDENTIALS=config/key.json

4. **Build the Docker Image:**
   docker-compose build

5. **Start the Airflow Scheduler & Web Server:**
   docker-compose up -d

- Once started, verify the running **Docker containers** with:
  docker ps

6. **Initialize Airflow Database & DAGs:**
   docker-compose up airflow-init

7. **Verify Airflow DAGs:**

- Open your browser and go to http://localhost:8080
- Login credentials for Airflow UI:
  Username: admin
  Password: admin
- Verify the following DAGs are listed in Airflow:

data_ingestion.py
data_preprocessing.py
schema_validator.py
anomalies.py
bias_detector.py

- Click on the "Trigger DAG" play button to start the execution.

8. To Stop & Restart the Pipeline:

- To stop all running containers:
  docker-compose down
- To restart the pipeline after stopping:
  docker-compose up -d

Airflow DAGs & Task Breakdown

DAG Task	Description
data_ingestion	Downloads dataset from GCS and saves it in data/raw/.
data_preprocessing	Cleans data, applies SMOTE, and saves as parquet.
schema_validator	Validates dataset against predefined schema.
bias_detector	Detects bias in sentiment distribution.
anomalies	Identifies anomalies and sends alerts if detected.

Tools & Technologies Used

MLOps Tools:

- **GitHub Actions** (CI/CD for automated deployment)
- **Docker** (Containerization for portability)
- **Apache Airflow** (Orchestration for workflow automation)
- **Google Cloud Storage (GCS)** (Data storage and access management)
- **Data Version Control (DVC)** (Version tracking of data & models)

Machine Learning & Data Processing:

- **Pandas, NumPy, SciPy** (Data manipulation)
- **Scikit-learn, Imbalanced-learn (SMOTE)** (Machine learning preprocessing)
- **TensorFlow Data Validation (TFDV)** (Schema validation & anomaly detection)

Docker and Airflow

Our data pipeline is containerized using Docker and orchestrated with Apache Airflow. The docker-compose.yaml file defines the required services, ensuring that all dependencies are pre-installed and configured. This approach guarantees platform independence, allowing the pipeline to run seamlessly on Windows, macOS, or Linux. By leveraging Docker, we eliminate environment inconsistencies, making the setup process more reproducible and scalable. Meanwhile, Apache Airflow manages workflow execution, ensuring that each step, from data ingestion to bias detection, it is automated and efficiently scheduled.

Data Version Control (DVC)

DVC (Data Version Control) is a powerful open-source tool designed to facilitate dataset management and versioning in machine learning workflows. It enables seamless tracking of data changes over time, ensuring reproducibility and efficient collaboration in ML projects. Unlike traditional version control systems, which primarily manage code, DVC stores metadata separately while keeping the actual datasets outside the Git repository, preventing unnecessary repository bloat.

By integrating effortlessly with Git, DVC helps teams manage large datasets, experiment tracking, and model versions in a structured manner. It ensures that every stage of the ML pipeline, from data preprocessing to model training, can be recreated consistently. With its robust approach to data versioning, DVC enhances workflow transparency, supports collaborative development, and maintains the integrity of machine learning experiments.

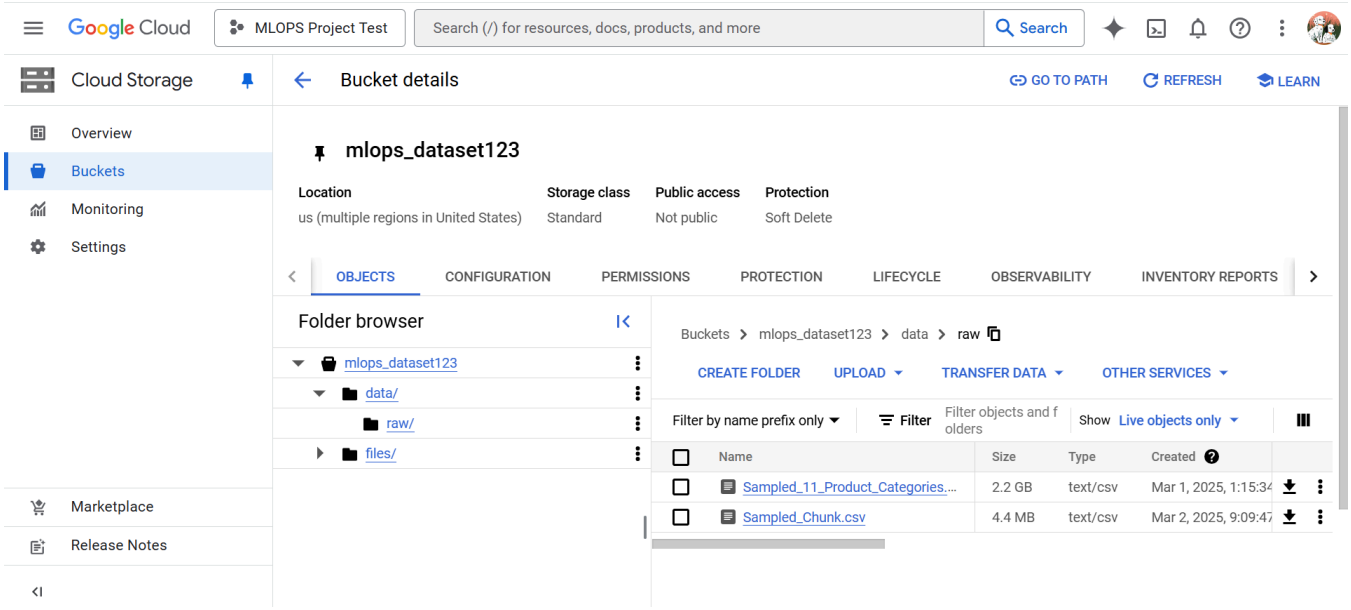
Google Cloud Platform (GCP) Setup

We leverage Google Cloud Storage (GCS) for securely storing and managing our machine learning models, ensuring seamless access for deployment and retrieval. To enable Google Cloud Platform (GCP) services for this project, follow these steps to set up a service account:

1. **Access Google Cloud Console:**
   Open Google Cloud Console.
2. **Navigate to Service Accounts:**
   In the left-hand menu, go to IAM & Admin > Service Accounts.
3. **Create a New Service Account:**
   Click "Create Service Account" and enter a name and description for easy identification.
4. **Assign Required Permissions:**
   Choose the necessary roles to grant appropriate access. You can select predefined roles (e.g., Storage Admin) or create a custom role based on project requirements.
5. **Generate and Download the Key:**
   After the service account is created, navigate to the "Keys" tab.
   Click "Add Key" > "Create new key".
   Choose JSON format and click Create to download the key file.
6. **Store the Key Securely:**
   Move the downloaded JSON key file to the config/ directory of your project.
   Ensure this key is referenced in the .env file for authentication.

By completing these steps, your environment will be configured to securely interact with Google Cloud Storage, allowing seamless data access and model storage.

Our MLOps Project - GCP bucket - You can avoid these steps of creating a GCP bucket instead, you could raise a request to access our GCP bucket.



Pictured above: GCP Bucket tracking the Data Version for the dataset

Pipeline Architecture

The sentiment analysis pipeline consists of the following stages:

1. Data Ingestion
2. Data Validation
3. Preprocessing & Cleaning
4. Bias & Anomaly Detection
5. Model Training & Evaluation
6. Deployment & Monitoring

Each component is automated using Apache Airflow for seamless execution.

Pipeline Components & Workflow

1. Data Ingestion (data_ingestion.py)

- Downloads raw Amazon customer review data from GCP Cloud Storage.
- Saves the raw dataset as reviews.csv under the data/raw/ directory.
- Uses Google Cloud SDK (gcsfs) for authentication.

```
fs = gcsfs.GCSFileSystem(token=SERVICE_ACCOUNT_KEY)
with fs.open(FILE_PATH) as f:
        df = pd.read_csv(f)
```

- Ensures that the dataset is downloaded from the Google cloud storage(GCS) bucket using service key authentication. It starts by loading environment variables that store configuration details such as the bucket name, source file path, and service account key. The script establishes a connection to GCS then attempts to download the specified file, and saves it locally. It also includes error handling to log any failures during the data retrieval process.

2. Data Validation (schema_validator.py)

- Checks for missing values, incorrect data types, and schema inconsistencies.
- Compares new dataset statistics with a predefined schema (schema.pbtxt).
- Generates an anomaly report if schema drift is detected.

```
schema = tfdv.load_schema_text(SCHEMA_PATH)
anomalies = tfdv.validate_statistics(stats, schema)
```

3. Data Preprocessing (data_preprocessing.py)

- Removes duplicates and null values.
- Encodes categorical variables (e.g., product_category).
- Scales star ratings (if needed) to ensure consistency.
- Applies SMOTE to handle class imbalance.
- Saves the cleaned dataset in Parquet & CSV formats.

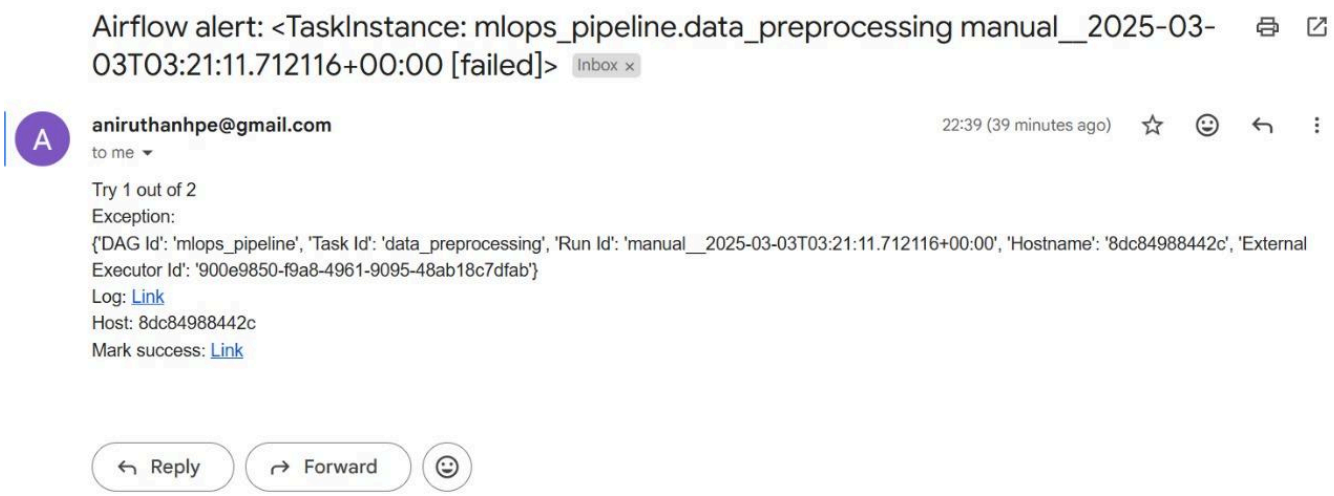
```
smote = SMOTE(sampling_strategy="auto", random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_numeric, y)
df_resampled = pd.concat([pd.DataFrame(X_resampled), pd.Series(y_resampled,
name="review_sentiment")], axis=1)
```

- Responsible for preparing raw data for machine learning by performing various preprocessing steps. It starts by setting up logging and ensuring that log files are stored correctly. The script reads raw data from a CSV file, and applies data transformations including cleaning, scaling numerical values using MinMaxScaler(), and encoding categorical features. It also addresses class imbalances using SMOTE (Synthetic Minority Over-sampling Technique) to balance the dataset. Finally, the processed data is saved in both Parquet and CSV formats for further analysis.

4. Anomaly Detection (anomalies.py)

- Uses TFDV to detect unusual data patterns or missing critical features.
- Generates an anomaly report and sends alerts if required.

```
anomalies = tfdv.validate_statistics(new_stats, schema)
if anomalies.anomaly_info:
        logging.warning(f" Anomalies detected: {anomalies}")
```



- The system is equipped with an email notification feature that promptly alerts the team whenever an anomaly is detected. These alerts provide detailed information about the identified issues, enabling the team to take swift corrective measures.

5. Bias Detection (bias_detector.py)

- Compares current dataset statistics with a reference dataset.
- Detects data drift to identify potential biases in sentiment distribution.
- Generates a bias report.

```
drift_results = []
for feature in new_stats.datasets[0].features:
        old_mean = ref_feature.num_stats.mean
        new_mean = feature.num_stats.mean
        percent_drift = ((new_mean - old_mean) / old_mean) * 100
        drift_results.append(f"{feature_name}: Drift = {percent_drift:.2f}%")
```

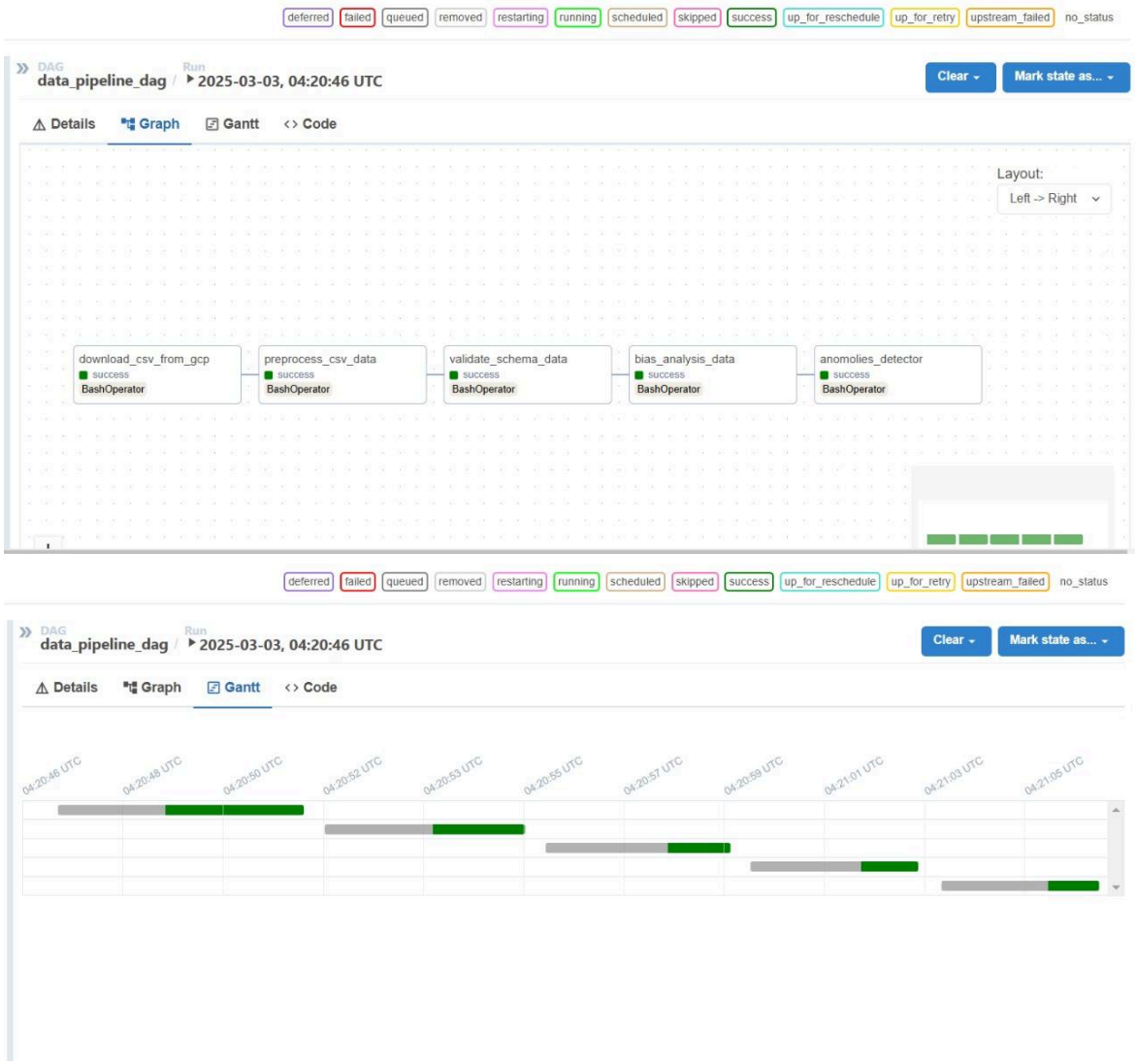
- Analyzes and detects biases in a processed dataset using TensorFlow Data Validation (TFDV). The script loads processed data from a Parquet file and utilizes predefined schema and reference statistics to compare new dataset statistics. It leverages TensorFlow Metadata to extract insights and detect biases based on statistical differences between datasets. The results include bias reports which are saved for future analysis.

Failure Handling & Alerts

- If schema anomalies, bias drift, or pipeline failures occur, an alert is sent via email.
- Automatic retraining is triggered when model drift exceeds a threshold.

```
AIRFLOW__SMTP__SMTP_HOST: smtp.gmail.com
AIRFLOW__SMTP__SMTP_USER: user@example.com
```

Pipeline Optimization



Pictured above: Airflow DAG Execution Graph and Gantt chart for Data Pipeline. It is a popular project management tool used to visualize and track the progress of tasks or activities over time. It provides a graphical representation of a pipeline's schedule, showing when each task is planned to start and finish.

Email Alerts

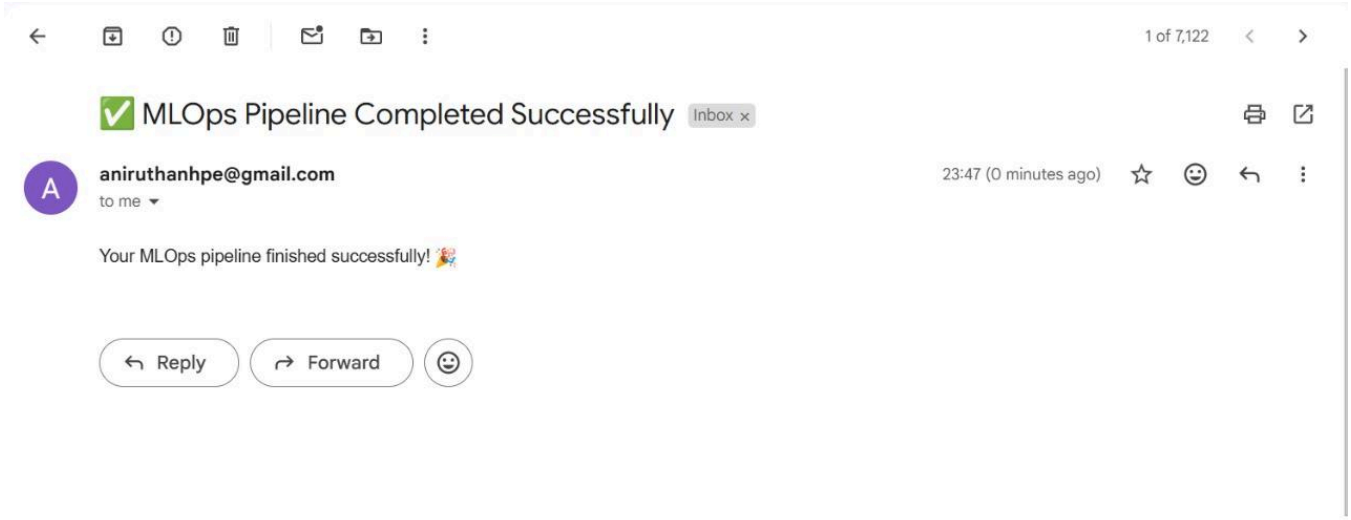
Email Notification Setup: The DAG is designed to send email alerts upon successful execution, enabling users to track task completion directly from their inbox. This feature ensures users remain informed about the pipeline's progress without needing to manually monitor the Airflow dashboard.

To enable email notifications, ensure the SMTP configurations are properly set in the docker-compose.yaml file under the [smtp] section. This should include specifying the SMTP server, sender email, recipient details, and authentication settings for seamless email delivery.

AIRFLOW_SMTP_SMTP_HOST: smtp.gmail.com AIRFLOW

SMTP_SMTP_STARTTLS: 'True' AIRFLOW_SMTP
SMTP_SSL: 'False'
AIRFLOW_SMTP_SMTP_USER: aniruthanhpe@gmail.com AIRFLOW_SMTP
*SMTP_PASSWORD: **** **** **** ****
AIRFLOW_SMTP SMTP_PORT: 587*
AIRFLOW_SMTP_SMTP_MAIL_FROM: aniruthanhpe@gmail.com AIRFLOW_SMTP
SMTP_TIMEOUT: 30 # SMTP timeout in seconds
AIRFLOW_SMTP_SMTP_RETRY_LIMIT: 5 # Maximum retries
AIRFLOW_SCHEDULER_ENABLE_HEALTH_CHECK: 'true'

Testing and Verification: After the DAG runs, confirm that a "DAG Completed" email is received in the specified inbox. This notification verifies that every task in the workflow has been successfully executed. Users also have the option to adjust the notification settings to trigger alerts for any task failures or retries, thereby strengthening the monitoring of critical workflows.



Pictured above: Dag completed notification sent to the recipient email.
Link to Github Repo:
https://github.com/Aniruthan-0709/Sentiment-Analysis-Tool-for-Product-Reviews